

CSE428: Image Processing

Lecture 5

Neighbourhood processing

Contents

- Spatial operations
- Neighborhood operations
- Spatial filtering
- Image padding
- Linear spatial filtering
- Correlation and convolution
- Smoothing spatial filters: Gaussian, Average & Median filtering
- Unsharp masking & high boost filtering
- Geometric transformations

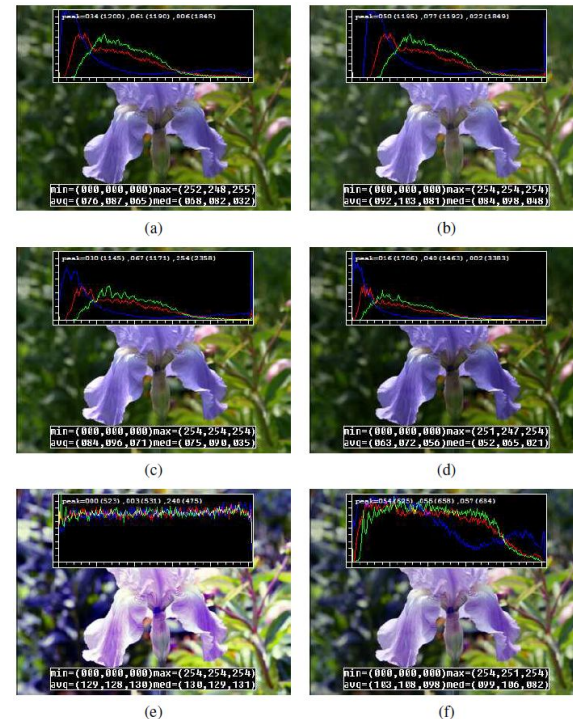
Spatial Operations

Can be subdivided into 3 broad categories:

1. Single-pixel operations or **point processing** (covered in the last lecture)
2. Neighborhood operations or **neighborhood processing**
3. Geometric spatial **transformations** (won't go into too much detail)

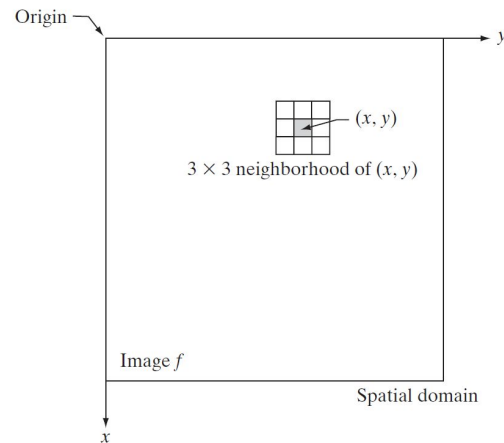
Single-pixel operations

- Simplest possible operation on a digital image: alter the pixel intensities
- If T is a transfer function, $s = T(z)$ or
 - $z \rightarrow$ original image pixel intensity $s \rightarrow$ mapped intensity
- Some local image processing operations:
 - (a) original image along with its three color histograms;
 - (b) brightness increased (additive offset, $b = 16$);
 - (c) contrast increased (multiplicative gain, $a = 1.1$);
 - (d) gamma (partially) linearized ($\gamma = 1.2$);
 - (e) full histogram equalization;
 - (f) partial histogram equalization



Neighborhood

- A pixel p at coordinates has four horizontal and vertical neighbors whose coordinates are given by
 - $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$, $(x, y - 1)$
 - This set of pixels, called the 4-neighbors of p , $N_4(p)$
- The four diagonal neighbors of p
 - $(x + 1, y + 1)$, $(x + 1, y - 1)$, $(x - 1, y + 1)$, $(x - 1, y - 1)$
 - Diagonal neighbors of p , $N_D(p)$
- 4-neighbors together with the diagonal neighbors are called the 8-neighbors of p : $N_8(p)$



Distance measures

- For pixels p , q , and z , with coordinates (x, y) , (s, t) , and (v, w)
- Euclidean distance between p and q
 - $D_e(p, q) = [(x - s)^2 + (y - t)^2]^{1/2}$
- D_4 distance (city-block distance) between p and q
 - $D_4(p, q) = |x - s| + |y - t|$
 - The pixels with $D_4 = 1$ are the 4-neighbors of (x, y)
- D_8 distance (chessboard distance) between p and q
 - $D_8(p, q) = \max(|x - s|, |y - t|)$
 - The pixels with $D_8 = 1$ are the 8-neighbors of (x, y)

			2		
		2	1	2	
2	1	0	1	2	
		2	1	2	
			2		

2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

Linear vs Nonlinear Operations

Consider a general operator, H such that : $H [f(x, y)] = g(x, y)$

H is said to be a linear operator if the following is true:

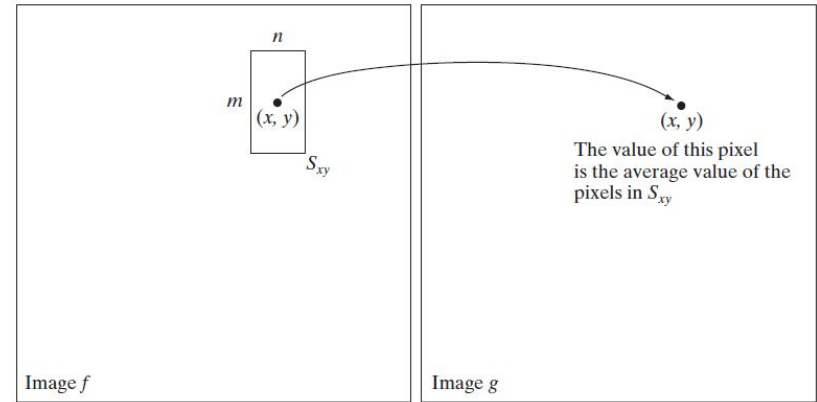
$$H [a_1 f_1(x, y) + a_2 f_2(x, y)] = a_1 H [f_1(x, y)] + a_2 H [f_2(x, y)]$$

1. *Additivity* property: output of a linear operation due to the sum of two inputs is the same as performing the operation on the inputs individually and then summing the results
2. *Homogeneity* property: the output of a linear operation to a constant times an input is the same as the output of the operation due to the original input multiplied by that constant

Neighborhood operations

S_{xy} is the set of coordinates of a neighborhood centered on an arbitrary point (x, y) in an image f

- *Neighborhood processing* generates a corresponding pixel at the same coordinates in an output (processed) image, g , such that the value of that pixel is determined by a specified operation involving the pixels in the input image with coordinates in S_{xy}



Digital Image Processing, Third Edition,
Rafael C. Gonzalez & Richard E. Woods

Neighborhood operations: Mechanism

Spatial filtering consists of

1. A **neighborhood** (defined by the filter / kernel, or mask)
2. A **predefined operation** (which is performed on the neighborhood)
 - Linear operation
 - Nonlinear operation

Filtering creates a new pixel at the same position of the center of the neighborhood, and whose value is the result of the filtering operation

A processed (filtered) image is generated as the center of the filter visits each pixel in the input image

Kernels (Also known as Filters/Masks)

- In image processing, a kernel (also known as a filter or convolution mask/matrix) is **a small matrix** used to apply effects like blurring, sharpening, edge detection, and more to an image.
- The kernel is applied to the image through a process called **convolution**, where the kernel is **systematically moved across the image, performing calculations to produce a modified image.**
- The process of filtering is also known as convolving a mask with an image. As this process is same of convolution so filter masks are also known as convolution masks.

Kernels (Also known as Filters/Masks)

- The general process of filtering and applying masks consists of moving the filter/mask from point to point in an image. At each point (x,y) of the original image, the response of a filter is calculated by a pre defined relationship. All the filters values are pre defined.
- Here are some common types of filters:
 1. Smoothing Filters (Low-pass filters)
 2. Sharpening Filters (High-pass filters)
 3. Edge Detection Filters
 4. Non-Linear Filters

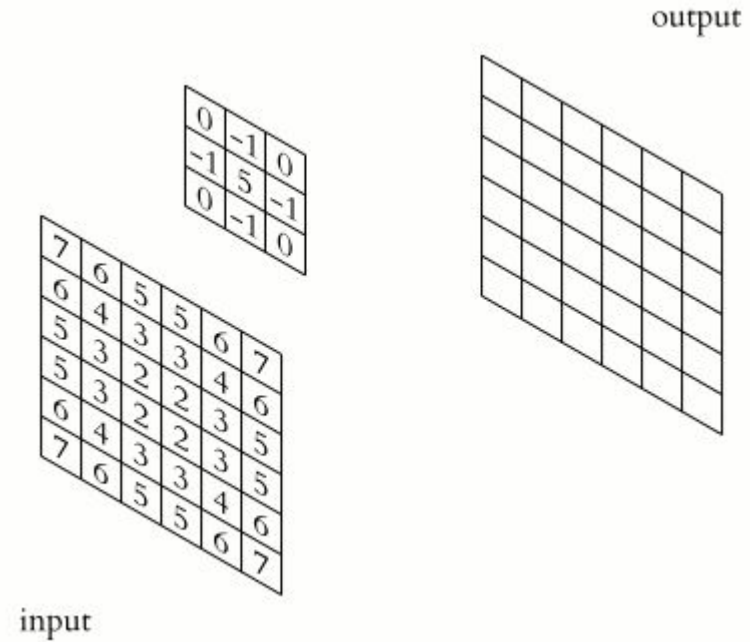
Key Concepts of Kernels

- Kernels are typically small matrices, such as 3x3, 5x5, or 7x7. The size of the kernel determines the area of the image that each calculation affects.
- The values within the kernel matrix determine the specific effect the kernel will have on the image. These values can be positive or negative and are used in weighted sums with the pixel values of the image.
- The kernel is placed on top of the image, covering a portion of it. Each element of the kernel is multiplied by the corresponding image pixel value. The results are summed to get the value of the new pixel in the output image. The kernel is then moved to the next position (typically one pixel to the right or down) and the process is repeated. This whole process is known as **Convolution**.

Key Concepts of Kernels

- The animation here demonstrates the process of filtering. Here, edge Padding was applied to preserve the dimensions of the input image.

- Kernel, $w(x, y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$

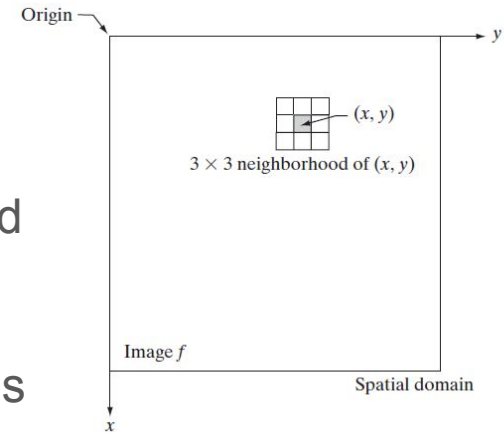


Spatial Filtering

Neighborhood processing is also called ***spatial filtering***

- $g(x, y) = T[f(x, y)]$
- Input image: $f(x, y)$
- Output image: $g(x, y)$
- $T[.]$ is an operator in f , defined over a neighborhood
 - Linear operation $\rightarrow$ linear spatial filtering
 - Nonlinear operation $\rightarrow$ nonlinear spatial filtering
- T can be applied to a single image or multiple images

Example: a 3x3 neighborhood of (x, y)



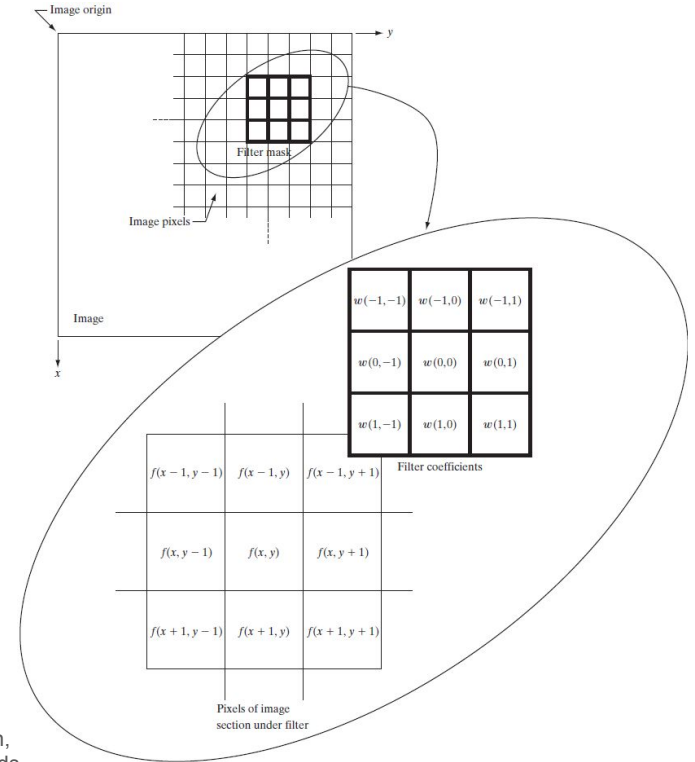
Linear Spatial Filtering

Linear spatial filtering in a $m \times n$ neighborhood

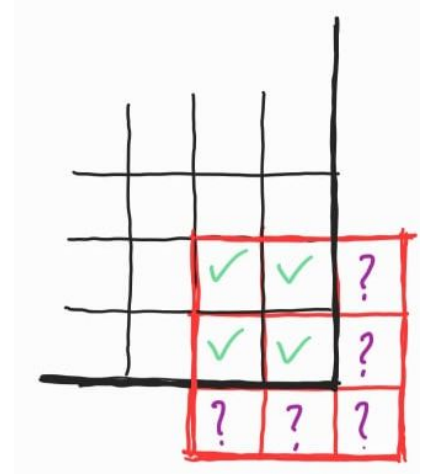
- Mask/filter/kernel size: $m \times n$
 - Assume $m = 2a + 1$, $n = 2b + 1$ (both odd numbers)
- Image size: $M \times N$

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

Example: $g(x, y) = w(-1, -1) f(x-1, y-1) + w(-1, 0) f(x-1, y) + \dots + w(0, 0) f(x, y) + \dots + w(1, 1) f(x+1, y+1)$



But.. what about the border pixels?



Solution 1: Don't visit the border pixels!
Result: Image is ***shrunk***

Solution 2: *Padding*

Result: Image shape *stays the same*

Image Padding

1. **Zero** padding: extend the borders of the original image with zeros
2. **Constant** padding: extend the borders of the original image with a constant pixel value
3. **Mirror** padding: extend the borders of the original image by reflecting the edge pixels
4. **Clamp/Edge/Nearest** padding: extend the borders of the original image by repeating edge pixels

More ways: <https://scikit-image.org/docs/dev/api/skimage.util.html#skimage.util.pad>

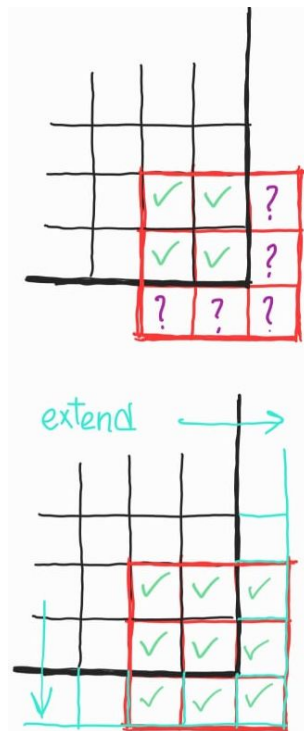
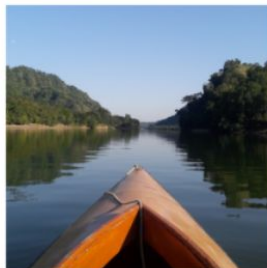


Image Padding

A 1448x1448x3 image padded with 200 pixels on each side:

1. Original image
2. Zero padding
3. Constant padding
4. Mirror padding
5. Edge padding
6. Linear(ramp) padding

1. Original image (no padding)



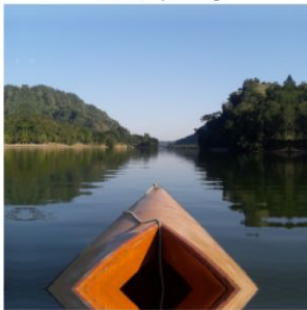
2. Zero padding (constant = 0)



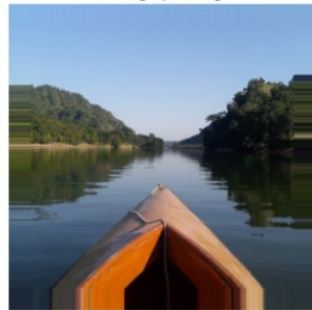
3. Constant padding (constant = 200)



4. Mirror padding



5. Edge padding



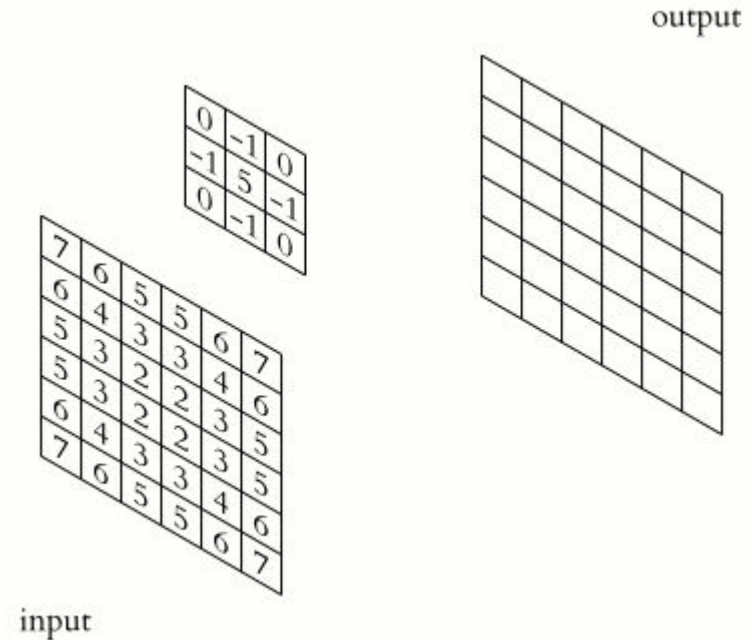
6. Linear ramp padding



Linear Spatial Filtering: Demo

- Kernel shape: 3x3
- Input image shape: 6x6
- Output image shape: 6x6
- Padding: Nearest
- Step (stride): 1

- Kernel, $w(x, y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$

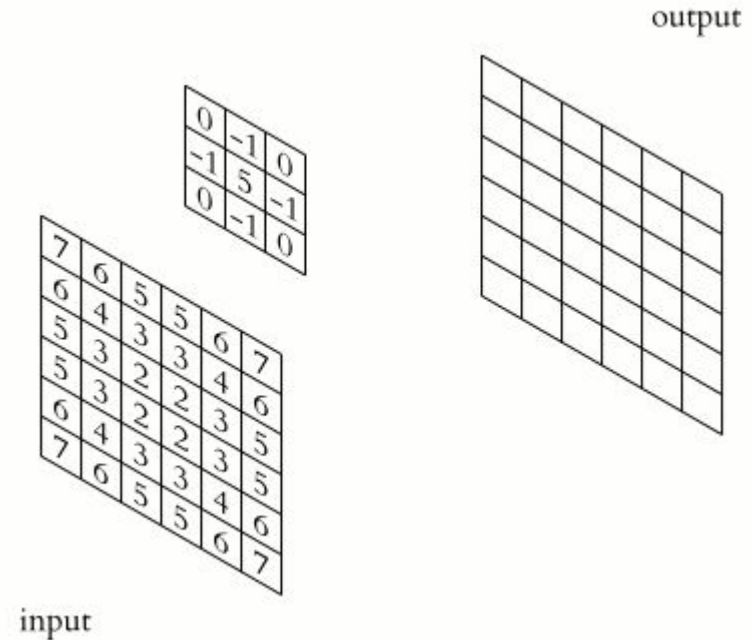


Linear Spatial Filtering: Output Shape

- Kernel shape: $m \times n$
- Input image shape: $M \times N$
- Number of pixels padded: p
- Stride: s
- Output image shape: $H \times W$

$$H = \text{floor}\left(\frac{M + 2p - m}{s} + 1\right)$$

$$W = \text{floor}\left(\frac{N + 2p - n}{s} + 1\right)$$



Correlation

Correlation between two 2D signals $w(x, y)$ & $f(x, y)$ is defined as:

$$w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x+s, y+t)$$

- Outputs a ***measure of similarity*** of the two signals as a function of the displacement of one relative to the other
- Also called the cross-correlation
- Basically a sliding dot product (vector element wise multiplication)
- Used for searching a long signal $[f(x, y)]$ for a shorter signal $[w(x, y)]$ (a known feature)

Linear spatial filtering $\Leftrightarrow$ 2D signal correlation

Correlation to Convolution

- Linear spatial filtering is basically a 2D signal **correlation** operation
- If we rotate the original filter $w(x, y)$ 180° and then proceed, it's called the **convolution** operation
- Mathematically,

$$\text{Correlation: } w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x+s, y+t)$$

$$\text{Convolution: } w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x-s, y-t)$$

For symmetric filters,
convolution and correlation both yield the
same result!

Smoothing Spatial Filters

- Linear
 - Box/average filtering
 - Gaussian filtering
- Non-linear
 - Median filtering

Example: A 3 x 3 filter with coefficients w_1 through w_9

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Linear Smoothing Filters

1. Averaging filter

- Average of all the pixels encompassed by the filter kernel
- An $m \times n$ averaging *box kernel*: $w(x, y) = 1 / m \times n$

2. Gaussian filter

- Weighted* average of all the pixels encompassed by the gaussian filter kernel
- An $m \times n$ *gaussian kernel*: $w(x, y) = \exp(-(x^2+y^2)/2\sigma^2)$

Example: a 3x3 Box filter and a 3x3 Gaussian filter

- (note that the filters are normalized so that the intensity of the output pixel remains unchanged)

$$\frac{1}{9} \times$$

1	1	1
1	1	1
1	1	1

$$\frac{1}{16} \times$$

1	2	1
2	4	2
1	2	1

Linear Smoothing Filters

Applications

- Image **Blurring** (As the name suggests)
- Image **Sharpening** (wait, sharpening from blurring? what?)
- Image **Denoising**
- **Low pass filtering** (getting rid of high variations)

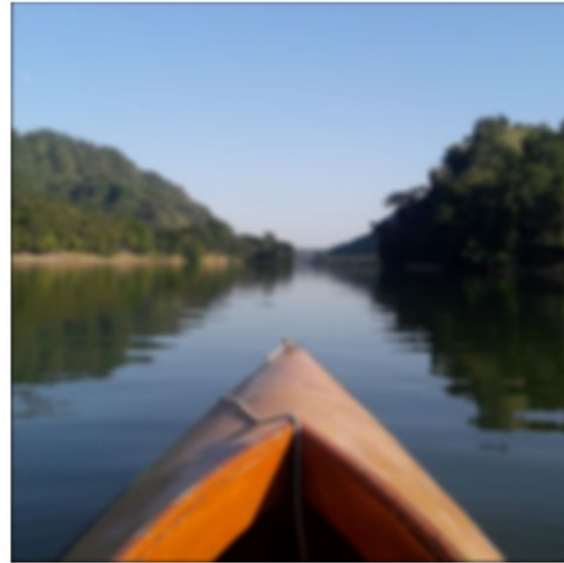
Gaussian Filtering: The Gaussian Blur

Result of filtering an image with with (25×25) , $\sigma^2 = 8$ Gaussian kernel

Original image



Gaussian blur



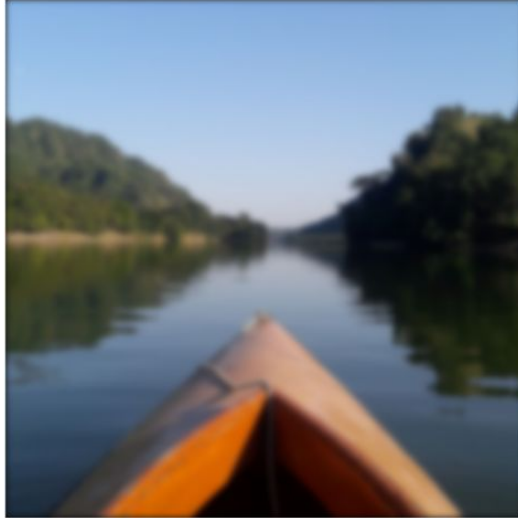
Gaussian Filtering: The Gaussian Blur

Increasing the kernel shape increases blurriness, borders become more prominent

(5 x 5)



(55 x 55)



(205 x 205)

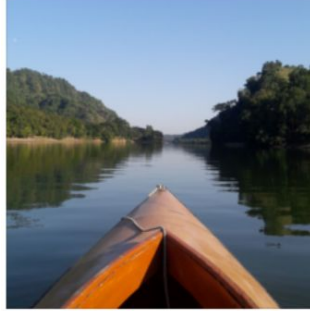


Gaussian Filtering vs Average Filtering

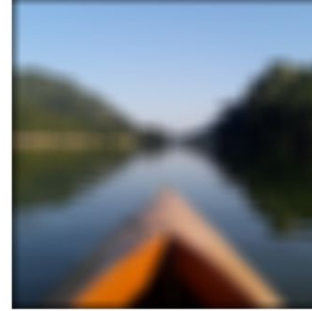
For the same kernel size the gaussian blur *preserves more detail* of the image

Even though the image is blurred the edges are somewhat better preserved for the gaussian blurring (edge preserving blurring)

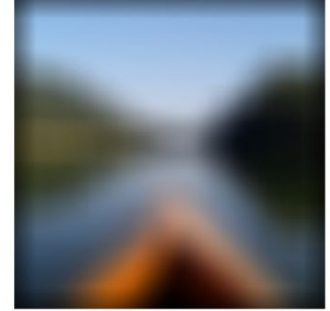
(5 x 5) Box kernel



(55 x 55) Box kernel



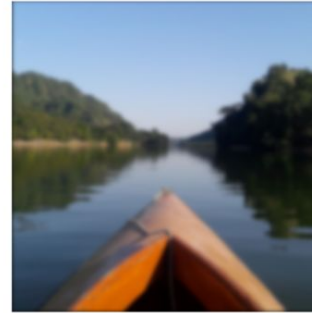
(205 x 205) Box kernel



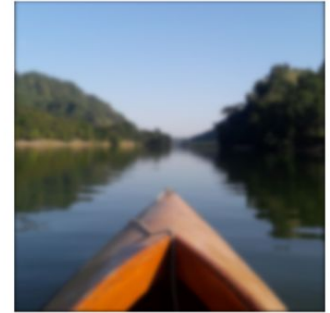
(5 x 5) Gaussian kernel



(55 x 55) Gaussian kernel



(205 x 205) Gaussian kernel



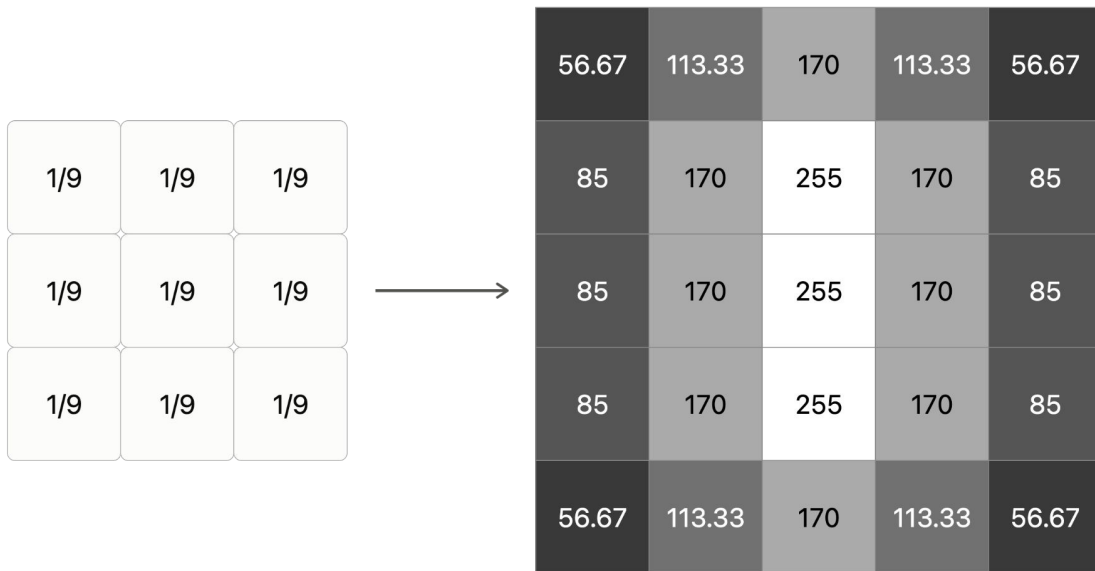
Filtering Example

Consider that we have this 5*5 image:

0	255	255	255	0
0	255	255	255	0
0	255	255	255	0
0	255	255	255	0
0	255	255	255	0

Box Filter

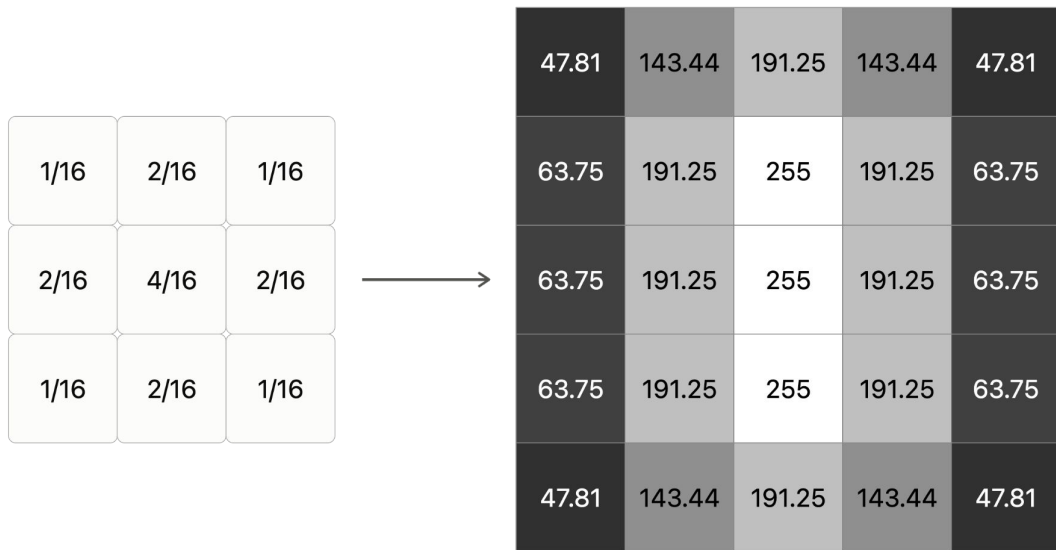
Output after applying Box Filter (Zero Padded)



Notice the corners have darkened the most (56.67) — that's zero-padding pulling values down where the window hangs off two edges at once.

Gaussian Filter

Output after applying Gaussian Filter (Zero Padded)



Same corner-darkening pattern, but consistently less severe than the box filter (e.g. 63.75 vs. 85 at the mid-edge, 47.81 vs. 56.67 at the corner) — the center-weighted kernel resists zero-padding's pull more than the uniform box average does.

Noise Reduction Example

0	0	0
0	255	0
0	0	0

Box vs Gaussian Filter (3*3, Zero Padding)



Can you notice the difference between two outputs?

Order Statistic Filtering

Response is based on *ordering* (ranking) the pixels contained in the pixel neighborhood (S_{xy})

- **Median** filtering:

$$\hat{f}(x, y) = \operatorname{median}_{(s,t) \in S_{xy}} \{g(s, t)\}$$

- **Max** and **min** filtering:

$$\hat{f}(x, y) = \max_{(s,t) \in S_{xy}} \{g(s, t)\} \quad \hat{f}(x, y) = \min_{(s,t) \in S_{xy}} \{g(s, t)\}$$

- **Midpoint** filtering:

$$\hat{f}(x, y) = \frac{1}{2} \left[\max_{(s,t) \in S_{xy}} \{g(s, t)\} + \min_{(s,t) \in S_{xy}} \{g(s, t)\} \right]$$

- **Alpha-trimmed** mean filtering: $\hat{f}(x, y) = \frac{1}{mn - d} \sum_{(s,t) \in S_{xy}} g_r(s, t)$

Median Filtering

Median Filter (Order-statistic filter)

- nonlinear spatial filter
- each pixel value is replaced by the median of the intensity values in the neighborhood
- excellent noise-reduction capabilities
- less blurring than linear filters
- particularly effective with impulse noise (salt and pepper noise)

Noise-corrupted Image



< Original
image (left)

Noisy image >
(right)
(Gaussian
noise)



Noise Reduction Using Linear Filtering



< Original
grayscale
image (left)

Noisy image >
(right) (Salt
and pepper
noise, pretty
common for
old b&w
cameras)



Noise Reduction Using Linear Filtering



Gaussian filter
used to
denoise both
noisy images
(25 x 25) $\sigma^2=3$

Output is
blurred, s&p
not properly
removed



Noise Reduction Using Nonlinear Filtering



Median filter is
used to
denoise the
salt and
pepper noise

(can you tell
the original
from the
filtered one?)



Noise Reduction using Median Filter

Suppose we have a constant region:

$$f = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 255 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

(255 is impulse noise)

Sorted neighborhood:

$$[10, 10, 10, 10, 10, 10, 10, 10, 255]$$

Median = 10

Impulse disappears completely.

This works because median is **nonlinear**.

Median filters remove salt and pepper noises. Salt noises indicate very high intensity values (white like salt grains) and pepper noises indicate very low intensity values (black like pepper).



Original cat image



Salt and pepper noise cat



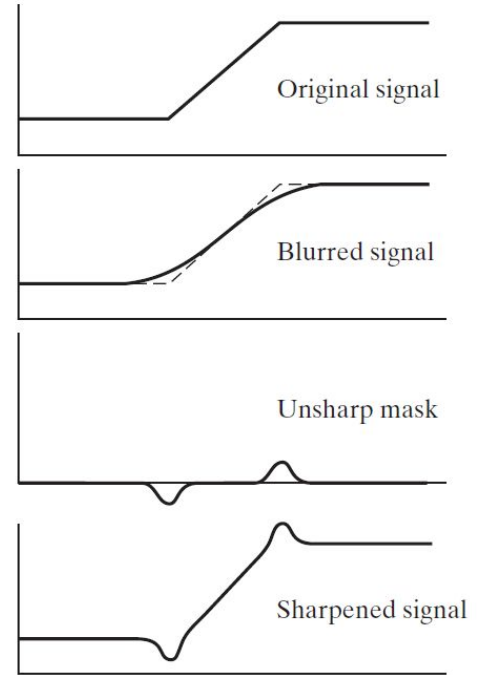
Median filtered cat

Unsharp Masking & High Boost Filtering

This unsharp masking process consists of the following steps:

1. Blur the original image
 - a. $f'(x, y) = \text{Blur}[f(x, y)]$
2. Subtract the blurred image from the original (the resulting difference is called the mask)
 - a. $g_{\text{mask}}(x, y) = f(x, y) - f'(x, y)$
3. Add the mask to the original
 - a. $g(x, y) = f(x, y) + k * g_{\text{mask}}(x, y)$

$k = 1$ is *unsharp masking*, $k > 1$ is *high boost filtering*



Unsharp Masking

Original image



Unsharp masking



High Boost Filtering

Original image



High boost filtering



Edge Detection

- Sudden changes of discontinuities in an image are known as edges. Significant transitions in an image are called as edges.
- Most of the shape information of an image is enclosed in edges.
- Often we detect these edges in an image by using filters and then by enhancing those areas of image which contain edges, we increase the sharpness of the image and the image becomes clearer.
- Some of the masks used for edge detection are: Prewitt Operator, Sobel Operator, Robinson Compass Masks, Krisch Compass Masks, Laplacian Operator.

Derivative Masks

- Edges are calculated by using difference between corresponding pixel intensities of an image. All the masks that are used for edge detection are also known as derivative masks.
- An image is a signal so changes in a signal can only be calculated using differentiation. That's why these operators are also called derivative operators or derivative masks.
- All the derivative masks should have the following properties:
 1. Opposite sign should be present in the mask.
 2. Sum of mask should be equal to zero.
 3. More weight means more edge detection.

Prewitt Operator

- Prewitt operator provides us with two separate masks. One for detecting edges in **horizontal** direction and another for detecting edges in an **vertical** direction.

- Mask for detecting edges in Vertical direction:

-1 0 1

-1 0 1

-1 0 1

- Mask for detecting edges in Horizontal direction:

-1 -1 -1

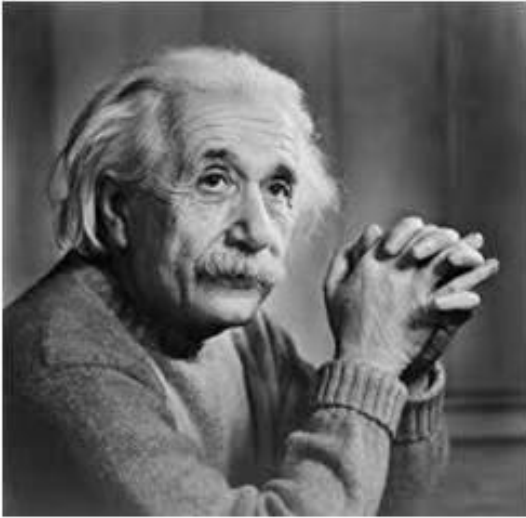
0 0 0

1 1 1

Prewitt Operator

- When we apply the vertical one it prominents the vertical edges. It simply works like as first order derivative and calculates the difference of pixel intensities in a edge region. As the center column is of zero, it does not include the original values of a pixel but rather it calculates the difference of right and left pixel values around that edge. This increases the edge intensity and it become enhanced comparatively to the original image.
- Similar thing happens for the horizontal one too. As the center row of the mask consists of zeros so it does not include the original pixel values of edges in the image, rather it calculates the difference of above and below pixel intensities of the particular edge. Thus increasing the sudden change of intensities and making the edge more visible. Both the above masks follow the principle of derivative mask. Both masks have opposite sign in them and both masks sum equals to zero.

Prewitt Operator



Original Image



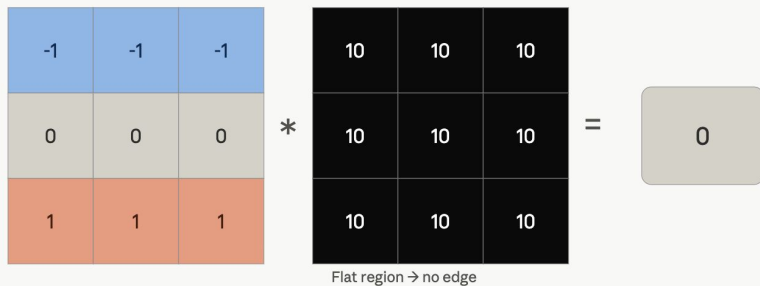
After applying Vertical
Mask



After applying
Horizontal Mask

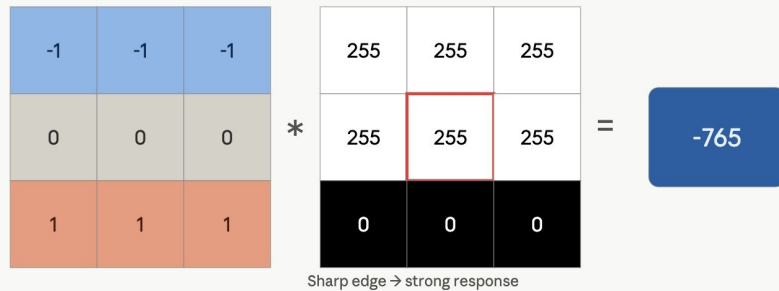
Prewitt Operator Example

Case 1 — uniform patch (no edge):



$-1(10) - 1(10) - 1(10) + 0 + 0 + 0 + 1(10) + 1(10) + 1(10) = -30 + 0 + 30 = 0$
— the top and bottom rows cancel because there's no vertical change in intensity.

Case 2 — sharp horizontal edge:



$-1(255) - 1(255) - 1(255) + 0 + 0 + 0 + 1(0) + 1(0) + 1(0) = -765 + 0 + 0 = -765$ — a large-magnitude result, so the mask flags this as an edge.

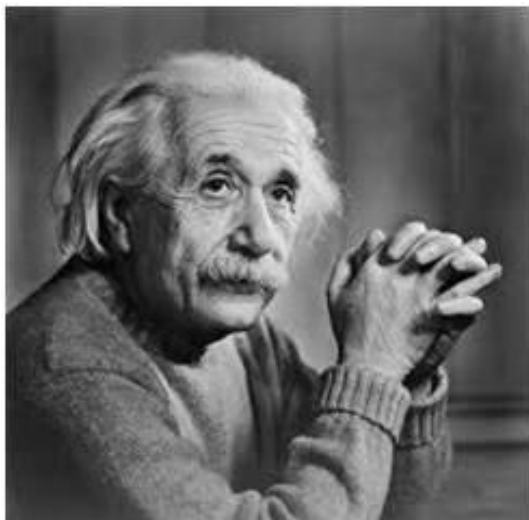
Sobel Operator

- Sobel operator also provides us with two separate masks. One for detecting edges in **horizontal** direction and another for detecting edges in an **vertical** direction.
- The Sobel operator usually incorporates a weighting of 2 in the middle row/column of the kernels. This weighted approach gives more emphasis to the central pixels, making the Sobel operator more **sensitive to edges** and **less sensitive to noise compared to the Prewitt operator**.
- Mask for detecting edges in Vertical direction:

-1	0	1
-2	0	2
-1	0	1
- Mask for detecting edges in Horizontal direction:

-1	-2	-1
0	0	0
1	2	1

Sobel Operator



Original Image



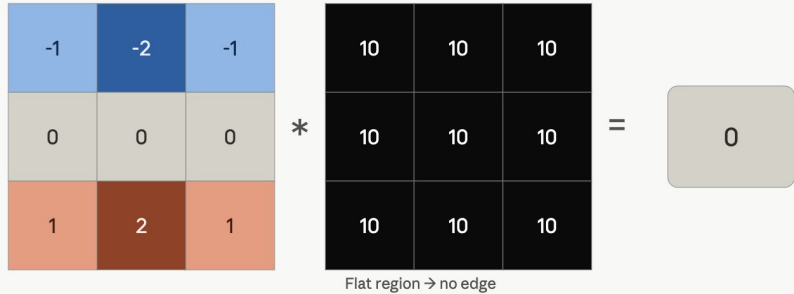
After applying Vertical
Mask



After applying
Horizontal Mask

Sobel Operator Example

Case 1 — uniform patch (no edge):



$$-1(10) - 2(10) - 1(10) + 0 + 0 + 0 + 1(10) + 2(10) + 1(10) = -40 + 0 + 40 = 0$$

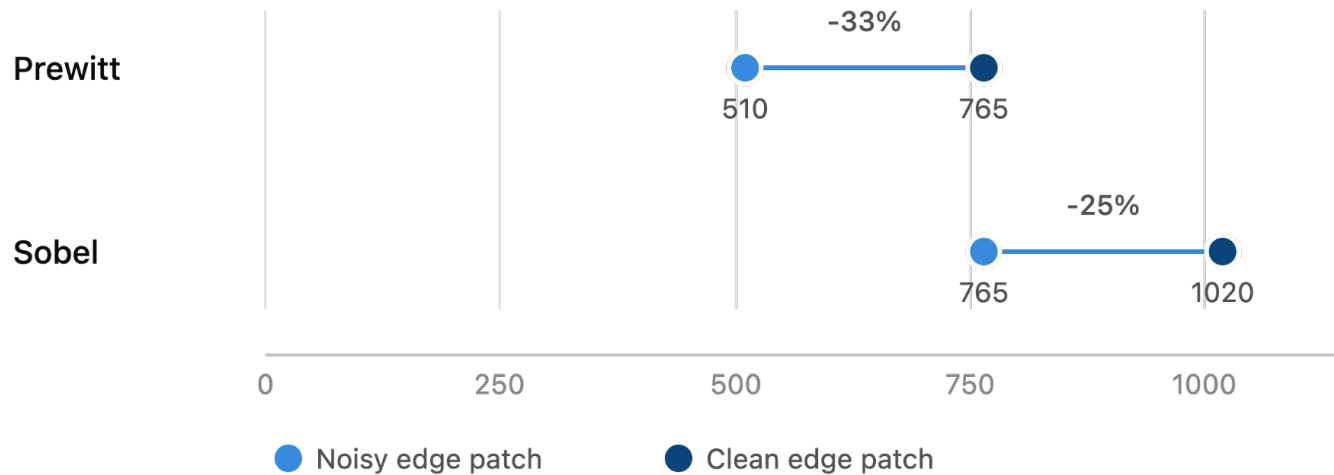
— still zero, as it must be for any flat region.

Case 2 — sharp horizontal edge:



$$-1(255) - 2(255) - 1(255) + 0 + 0 + 0 + 1(0) + 2(0) + 1(0) = -1020 + 0 + 0 = -1020$$

Prewitt vs Sobel



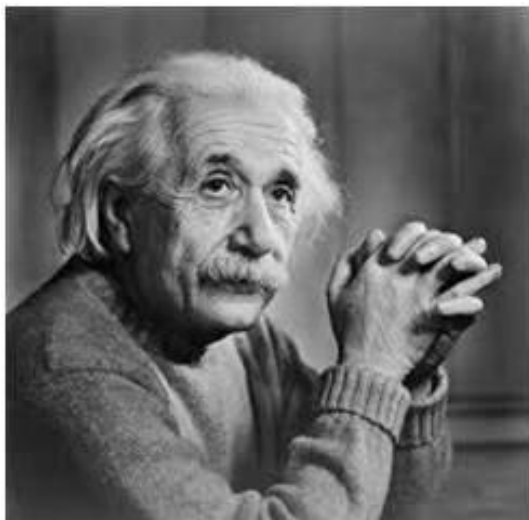
Robinson Compass Masks

- Robinson compass mask is another type of derivative mask which is used for edge detection. This operator is also known as direction mask. In this operator we take one mask and rotate it in all the 8 compass major directions that are following: *North, North West, West, South West, South, South East, East, North East*. Each mask gives us the edges on its direction.
- Ex.: Mask for detecting edges in North West direction:

0	1	2
-1	0	1
-2	-1	0
- Ex.: Mask for detecting edges in East direction:

-1	-2	-1
0	0	0
1	2	1

Robinson Compass Masks



Original Image



After applying Mask
for North West
Direction



After applying Mask
for South East
Direction

Laplacian Operator

- Laplacian Operator is also a derivative operator which is used to find edges in an image. The major difference between Laplacian and other operators like Prewitt, Sobel, Robinson and Kirsch is that these all are first order derivative masks but Laplacian is a second order derivative mask. In this mask we have two further classifications one is Positive Laplacian Operator and other is Negative Laplacian Operator.
- Another difference between Laplacian and other operators is that unlike other operators Laplacian didn't take out edges in any particular direction but it take out edges in following classification.
 1. Inward Edges
 2. Outward Edges

Laplacian Operator

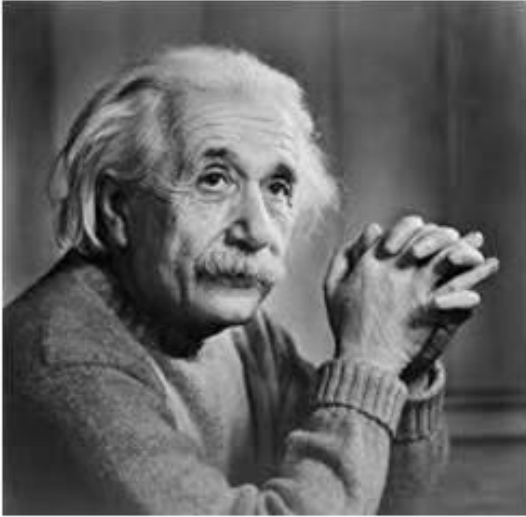
- Positive Laplacian Operator:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

- Negative Laplacian Operator:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Laplacian Operator



Original Image



After applying
Positive Laplacian
Operator



After applying
Negative Laplacian
Operator

Geometric Transformations

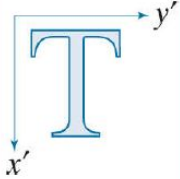
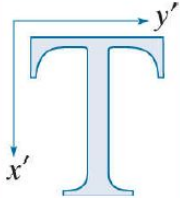
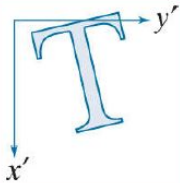
- We use geometric transformations modify the spatial arrangement of pixels in an image
- These transformations are called rubber-sheet transformations because they may be viewed as analogous to “printing” an image on a rubber sheet, then stretching or shrinking the sheet according to a predefined set of rules

Geometric Transformations

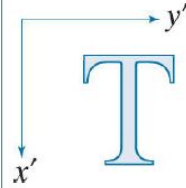
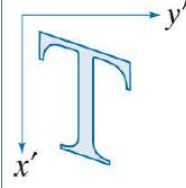
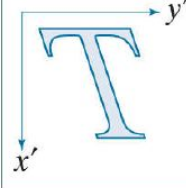
- (x, y) are pixel coordinates in the original image
- (x', y') are the corresponding pixel coordinates of the transformed image
- Our interest is in so-called affine transformations, which include scaling, translation, rotation, and shearing
- The key characteristic of an affine transformation in 2-D is that it preserves points, straight lines, and planes
-

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{A} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Geometric Transformations

Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x \\ y' &= y \end{aligned}$	
Scaling/Reflection (For reflection, set one scaling factor to -1 and the other to 0)	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= c_x x \\ y' &= c_y y \end{aligned}$	
Rotation (about the origin)	$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x \cos\theta - y \sin\theta \\ y' &= x \sin\theta + y \cos\theta \end{aligned}$	

Geometric Transformations

Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Translation	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + t_x$ $y' = y + t_y$	
Shear (vertical)	$\begin{bmatrix} 1 & s_v & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + s_v y$ $y' = y$	
Shear (horizontal)	$\begin{bmatrix} 1 & 0 & 0 \\ s_h & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = s_h x + y$	

Tutorial

Basic python implementation of some of the algorithms taught in class can be found in the following google colab notebook:

<https://colab.research.google.com/drive/1JF3ww3I-bBzoC0iy9Ado7eQLcxC1hVFU?usp=sharing>

Thank you!